

here we have a signal called *haveFortune()*, which becomes the property *onHaveFortune* inside the QML component, and gets assigned an event handler in the QML file, as follows:

```
onHaveFortune: mainFortuneLabel.setText(fortune);
```

We can now take a look at the C++ code for the object:

```
#include "fortuneclient.h"

FortuneClient::FortuneClient()
{
    mPort = 0;
    clientSocket = new QTcpSocket(this);
}

FortuneClient::~FortuneClient()
{
    clientSocket->disconnectFromHost();
    clientSocket->deleteLater();
}

/* accessors and mutators for the properties go first */

QString FortuneClient::serverHost() const
{
    return mHost;
}

void FortuneClient::setServerHost(QString host)
{
    mHost = host;
}

quint16 FortuneClient::serverPort() const
{
    return mPort;
}

void FortuneClient::setServerPort(int port)
{
    mPort = (quint16)port;
}

/* the slot we call to get a new fortune */

void FortuneClient::getNewFortune()
{
    QByteArray fortune;
    clientSocket->connectToHost(mHost, mPort,
    QIODevice::ReadOnly);
    clientSocket->waitForReadyRead();
    fortune = clientSocket->readAll();
    clientSocket->disconnectFromHost();
```

```
emit haveFortune(QString(fortune));
}
```

You already know how we're using this inside QML.

Firing it up

We'll need to write *main.cpp* first, which is luckily less than five lines of actual code. Here it is:

```
#include <QCoreApplication>
#include "fortuneclient.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    QCoreApplicationEngine engine;
    qmlRegisterType<FortuneClient>("Fortune", 1, 0,
    "FortuneClient");
    engine.load(QUrl(QStringLiteral("qrc:/main.qml")));
    return app.exec();
}
```

The critical line here is the *qmlRegisterType()* statement, which registers a component and injects it into the *QCoreApplicationEngine*. The syntax is:

```
qmlRegisterType<object_name>("namespace", major_version,
minor_version, "component_name");
```

So for the above line, you'd import the Fortune 1.0 name space, and create a FortuneClient object within the QML code.

You can now go ahead and build the application, and run it. Run the server that we built last month, and in the GUI, go to *Application->Set Server Details*; fill in the host (127.0.0.1) and the port (56789), and hit OK. The status bar should now say OK: *Server set to 127.0.0.1:56789*. Go ahead and hit the *Get Fortune* button. If everything works fine, you'll be greeted with your fortune cookie. 

References

- [1] The QML Documentation
- <http://doc.qt.io/qt-5/qtqml-index.html>
- [2] The QML Complete Reference
- <http://doc.qt.io/qt-5/qmlreference.html>
- [3] The QtQuick Documentation
- <http://doc.qt.io/qt-5/qtquick-index.html>
- [4] The QtQuick QML Types Reference
- <http://doc.qt.io/qt-5/qtquick-qmlmodule.html>

By: Boudhayan Gupta

Describing himself as a 'retard by choice', the author believes that madness is a cure-all for whatever is wrong or right with society. A social media enthusiast, he can be reached at @BaloneyGeek on Twitter.